

KAN-23 – Optimización de consultas SQL (Base de datos mentorías)

Índice

KAN-23 – Optimización de consultas SQL (Base de datos mentorías).....	1
Índice.....	1
1. Objetivo.....	1
2. Descripción de tablas.....	1
Tabla usuarios.....	1
Tabla perfiles.....	2
Relación entre tablas.....	2
3. Problema identificado.....	2
4. Estrategia de optimización.....	2
5. Consultas optimizadas.....	2
5.1 Consulta base (ineficiente).....	3
5.2 Consulta optimizada general.....	3
5.3 Búsqueda filtrada por carrera.....	3
5.4 Consulta con filtro avanzado.....	4
5.5 Listado rápido optimizado.....	4
6. Índices propuestos.....	4
7. Conclusión.....	5

1. Objetivo

Optimizar las consultas SQL relacionadas con la búsqueda de mentores en la base de datos mentorías, mejorando el rendimiento mediante la reducción de datos innecesarios, el uso adecuado de JOINS y la implementación de índices para acelerar las búsquedas.

2. Descripción de tablas

Tabla usuarios

Almacena la información principal de los usuarios del sistema.

Campos relevantes:

- id: identificador único del usuario
- nombre: nombre del usuario
- email: correo electrónico
- rol: tipo de usuario (en este caso, 2 representa mentores)
- estado: estado del usuario (activo/inactivo)

Tabla perfiles

Almacena información adicional del mentor.

Campos relevantes:

- usuario_id: relación con la tabla usuarios
- carrera: área de estudio
- ciclo: ciclo académico
- habilidades: habilidades del usuario
- disponibilidad: horarios disponibles

Relación entre tablas

`usuarios.id = perfiles.usuario_id`

3. Problema identificado

Se identificaron los siguientes problemas en las consultas iniciales:

- Uso de SELECT * que genera carga innecesaria de datos
- Consultas poco eficientes al traer información no utilizada
- Filtros aplicados después del JOIN sin optimización previa

- Falta de índices para acelerar búsquedas frecuentes
 - Consultas no optimizadas para filtrado por carrera, ciclo y estado
-

4. Estrategia de optimización

La optimización se realizó mediante:

- Selección de columnas específicas en lugar de SELECT *
 - Aplicación de filtros directos en la consulta (WHERE)
 - Uso adecuado de INNER JOIN para relaciones controladas
 - Reducción de datos innecesarios en consultas de listado
 - Propuesta de índices para mejorar el rendimiento en búsquedas frecuentes
-

5. Consultas optimizadas

5.1 Consulta base (ineficiente)

Consulta original que sirve como punto de comparación:

```
SELECT *  
FROM usuarios u  
JOIN perfiles p ON p.usuario_id = u.id  
WHERE u.rol = 2;
```

5.2 Consulta optimizada general

Se eliminan columnas innecesarias y se filtra por usuarios activos.

```
SELECT  
  u.id,  
  u.nombre,  
  u.email,  
  u.estado,  
  p.carrera,  
  p.ciclo,  
  p.disponibilidad  
FROM usuarios u  
INNER JOIN perfiles p ON p.usuario_id = u.id  
WHERE u.rol = 2  
AND u.estado = 'activo';
```

5.3 Búsqueda filtrada por carrera

Optimizada para búsquedas por área de estudio.

```
SELECT
  u.id,
  u.nombre,
  p.carrera,
  p.ciclo,
  p.disponibilidad
FROM usuarios u
INNER JOIN perfiles p ON p.usuario_id = u.id
WHERE u.rol = 2
AND u.estado = 'activo'
AND p.carrera = 'Software';
```

5.4 Consulta con filtro avanzado

Permite filtrar por nivel académico.

```
SELECT
  u.id,
  u.nombre,
  p.carrera,
  p.ciclo,
  p.habilidades,
  p.disponibilidad
FROM usuarios u
INNER JOIN perfiles p ON p.usuario_id = u.id
WHERE u.rol = 2
AND u.estado = 'activo'
AND p.ciclo >= 5;
```

5.5 Listado rápido optimizado

Diseñado para mostrar resultados en interfaces de usuario.

```
SELECT
  u.id,
  u.nombre,
  p.carrera
FROM usuarios u
INNER JOIN perfiles p ON p.usuario_id = u.id
```

```
WHERE u.rol = 2  
AND u.estado = 'activo'  
LIMIT 20;
```

6. Índices propuestos

Se proponen los siguientes índices para mejorar el rendimiento de las consultas:

```
CREATE INDEX idx_perfiles_usuario_id  
ON perfiles(usuario_id);
```

```
CREATE INDEX idx_usuarios_rol  
ON usuarios(rol);
```

```
CREATE INDEX idx_usuarios_estado  
ON usuarios(estado);
```

```
CREATE INDEX idx_perfiles_carrera  
ON perfiles(carrera);
```

```
CREATE INDEX idx_usuarios_rol_estado  
ON usuarios(rol, estado);
```

7. Conclusión

Se logró optimizar las consultas SQL mediante la eliminación de operaciones innecesarias como SELECT *, el uso de filtros directos en las consultas, la optimización de JOINS entre tablas relacionadas y la propuesta de índices estratégicos.

Estas mejoras permiten reducir la carga de procesamiento en la base de datos y mejorar el tiempo de respuesta en búsquedas de mentores, especialmente en escenarios con gran volumen de datos.